

Résumé Exécutif

Ce document présente les perspectives d'évolution du système vers une architecture **multi-tenant**, permettant de transformer un système de recommandation spécialisé en une **plateforme universelle** capable de servir simultanément plusieurs domaines d'application.

Le système actuel est un moteur de recommandation performant, spécialisé dans le domaine des véhicules. Cette spécialisation, bien qu'efficace, limite son potentiel d'exploitation à un seul cas d'usage. L'objectif de cette évolution est de capitaliser sur l'infrastructure existante pour créer un service de **Recommandation-as-a-Service (RaaS)** capable de servir n'importe quel domaine métier : immobilier, restauration, e-commerce, et bien d'autres.

L'architecture multi-tenant proposée repose sur trois piliers fondamentaux :

- **Isolation complète des données** : Chaque plateforme cliente dispose de son propre espace de stockage sécurisé
- **Scoring personnalisable** : Les critères de recommandation sont configurables par domaine
- **Infrastructure partagée** : Un seul système à maintenir pour de multiples applications

🔑 **Mots-clés** : système de recommandation, architecture multi-tenant, intelligence artificielle, embeddings vectoriels, scoring personnalisable, RaaS

Table des matières

Résumé Exécutif	1
1 Contexte et Problématique	6
1.1 Situation Actuelle	6
1.1.1 Limitations Identifiées	6
1.1.2 Analogie Explicative	6
2 Solution Proposée : Architecture Multi-Tenant	8
2.1 Vision Globale	8
2.2 Définition du Concept de Tenant	8
3 Fonctionnement du Système	10
3.1 Flux de Traitement d'une Recommandation	10
3.1.1 Détail des Étapes	10
3.2 Concept d'Embedding Vectoriel	11
4 Architecture Technique	12
4.1 Vue d'Ensemble des Composants	12
4.2 Description des Composants	13
5 Isolation et Sécurité des Données	14
5.1 Importance de l'Isolation	14
5.2 Stratégie d'Isolation par Schéma	14
6 Stratégies de Modèles d'Intelligence Artificielle	16
6.1 Approche A : Modèle Partagé	16
6.2 Approche B : Modèles Spécialisés	16
6.3 Recommandation Stratégique	17
7 Mécanisme de Scoring Personnalisable	18
7.1 Le Défi de la Personnalisation	18
7.2 Configuration des Poids de Scoring	18
7.3 Calcul du Score Final	19
8 Avantages et Bénéfices	20

8.1	Pour l'Entreprise	20
8.2	Pour les Plateformes Clientes	20
8.3	Pour les Utilisateurs Finaux	21
9	Feuille de Route d'Implémentation	22
9.1	Planning Prévisionnel	22
9.2	Détail des Phases	22
	Conclusion	23
	Glossaire	24

Table des figures

Figure 1.1	Architecture mono-domaine actuelle	6
Figure 2.1	Vision du système Multi-Tenant	8
Figure 3.1	Flux de traitement d'une recommandation	10
Figure 3.2	Principe de l'embedding vectoriel	11
Figure 4.1	Architecture technique du système multi-tenant	12
Figure 5.1	Isolation des données par schéma PostgreSQL	14
Figure 6.1	Approche A : Un modèle partagé pour tous les tenants	16
Figure 6.2	Approche B : Modèles spécialisés par domaine	16
Figure 7.1	Exemple de configuration des poids pour le domaine Véhicules	18
Figure 9.1	Feuille de route en trois phases	22

Liste des tableaux

Tableau 1.1	Limitations de l'architecture actuelle	6
Tableau 2.1	Exemples de tenants et leurs domaines	9
Tableau 3.1	Description détaillée des étapes du flux	10
Tableau 4.1	Description et analogie des composants	13
Tableau 5.1	Objectifs de l'isolation des données	14
Tableau 6.1	Avantages et inconvénients de l'approche modèle partagé	16
Tableau 6.2	Avantages et inconvénients de l'approche modèles spécialisés	17
Tableau 7.1	Critères de qualité par domaine	18
Tableau 8.1	Bénéfices pour l'entreprise	20
Tableau 8.2	Bénéfices pour les plateformes clientes	20
Tableau 8.3	Bénéfices pour les utilisateurs finaux	21
Tableau 9.1	Détail des phases d'implémentation	22
Tableau 9.2	Glossaire des termes techniques	24

Contexte et Problématique

1.1 Situation Actuelle

Le système est actuellement un moteur de recommandation **mono-domaine**, conçu exclusivement pour la recommandation de véhicules. Bien que performant dans son domaine de spécialisation, cette architecture présente des limitations significatives.

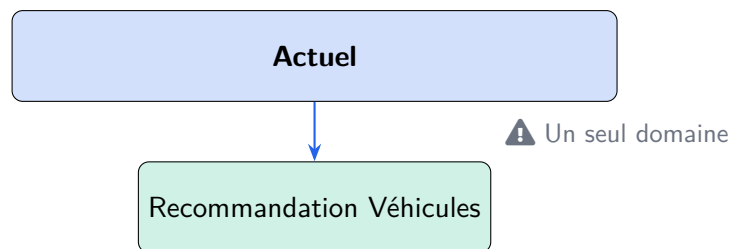


Figure 1.1. Architecture mono-domaine actuelle

1.1.1 Limitations Identifiées

Limitation	Impact
Mono-domaine	Nécessité de développer un nouveau système pour chaque nouveau domaine
Duplication	Multiplication des coûts de maintenance et d'infrastructure
Incohérence	Difficulté à maintenir des standards de qualité uniformes
Évolutivité	Impossibilité de partager les améliorations entre domaines

Table 1.1. Limitations de l'architecture actuelle

1.1.2 Analogie Explicative

💡 **Pour mieux comprendre :** Imaginons un chef cuisinier qui ne sait préparer que des pizzas. Si un client souhaite des sushis, il faut engager un autre chef, avec son propre équipement et sa propre cuisine. Cette approche est coûteuse et inefficace.

L'idéal : Un chef polyvalent capable de préparer n'importe quel plat selon la demande, dans une cuisine bien équipée.

Solution Proposée : Architecture Multi-Tenant

2.1 Vision Globale

La transformation en système multi-tenant vise à créer une plateforme de **Recommendation-as-a-Service (RaaS)** centralisée, capable de servir simultanément plusieurs plateformes clientes avec une isolation totale des données.

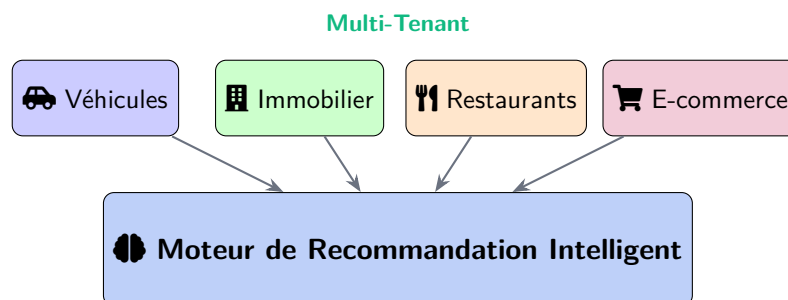


Figure 2.1. Vision du système Multi-Tenant

2.2 Définition du Concept de Tenant

Qu'est-ce qu'un Tenant ?

Un **tenant** (locataire) représente une plateforme cliente du système. Chaque tenant :

- Possède ses propres données (produits, avis clients)
- Définit ses propres critères de recommandation
- Est complètement isolé des autres tenants
- Accède au système via une API authentifiée

Tenant	Domaine	Exemple de Recommandation
Véhicules	Automobile	Voitures selon budget et préférences
Restaurants	Restauration	Établissements selon goûts et localisation
Immobilier	Logement	Appartements selon critères et budget
E-commerce	Commerce	Produits selon historique d'achat

Table 2.1. Exemples de tenants et leurs domaines

Fonctionnement du Système

3.1 Flux de Traitement d'une Recommandation

Le processus de recommandation suit un flux en six étapes, de la saisie utilisateur jusqu'à la présentation des résultats personnalisés.

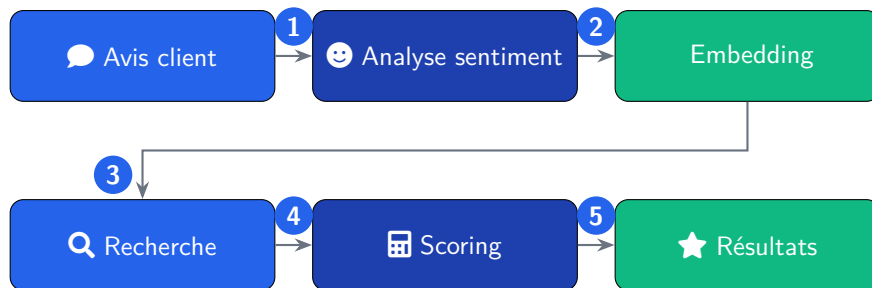


Figure 3.1. Flux de traitement d'une recommandation

3.1.1 Détail des Étapes

N°	Étape	Description
1	Entrée utilisateur	Le client saisit sa recherche ou son avis
2	Analyse de sentiment	L'IA détermine si l'avis est positif, négatif ou neutre
3	Génération d'embedding	Le texte est transformé en vecteur numérique (empreinte digitale du texte)
4	Recherche vectorielle	Identification des produits ayant une empreinte similaire
5	Calcul du scoring	Application des critères pondérés de la plateforme
6	Présentation	Affichage des recommandations personnalisées

Table 3.1. Description détaillée des étapes du flux

3.2 Concept d'Embedding Vectoriel

i Qu'est-ce qu'un embedding ?

Un **embedding** est une représentation mathématique d'un texte sous forme de vecteur de nombres. Cette technique permet de capturer le **sens sémantique** du texte, de sorte que des textes similaires en signification auront des vecteurs proches dans l'espace mathématique.



Figure 3.2. Principe de l'embedding vectoriel

Architecture Technique

4.1 Vue d'Ensemble des Composants

L'architecture multi-tenant repose sur plusieurs composants interconnectés, chacun jouant un rôle spécifique dans le traitement des recommandations.

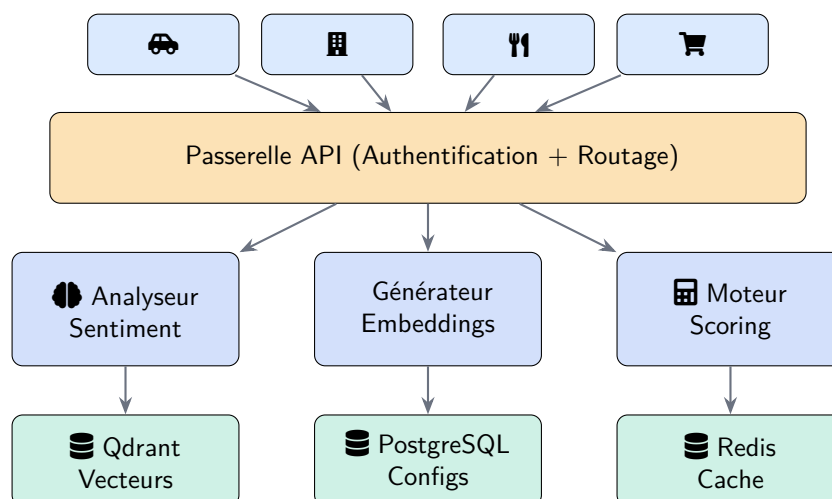


Figure 4.1. Architecture technique du système multi-tenant

4.2 Description des Composants

Composant	Rôle	Analogie
Passerelle API	Point d'entrée unique	Réceptionniste d'hôtel
Analyseur de Senti- ment	Comprend les émotions	Psychologue à l'écoute
Générateur d'Em- beddings	Transforme texte en nombres	Traducteur universel
Moteur de Scoring	Calcule les scores finaux	Juge avec des critères
Qdrant	Stocke les vecteurs	Bibliothèque ultra-rapide
PostgreSQL	Stocke les configura- tions	Classeur organisé
Redis	Cache les résultats	Mémoire à court terme

Table 4.1. Description et analogie des composants

Isolation et Sécurité des Données

5.1 Importance de l'Isolation

L'isolation des données entre tenants est un aspect critique de l'architecture multi-tenant. Elle garantit la sécurité, la confidentialité et l'intégrité des données de chaque plateforme cliente.

Icône	Objectif	Description
🔒	Sécurité	Les données d'un tenant sont inaccessibles aux autres
🧹	Propreté	Aucun mélange entre domaines différents
📈	Performance	Chaque plateforme est optimisée indépendamment

Table 5.1. Objectifs de l'isolation des données

5.2 Stratégie d'Isolation par Schéma

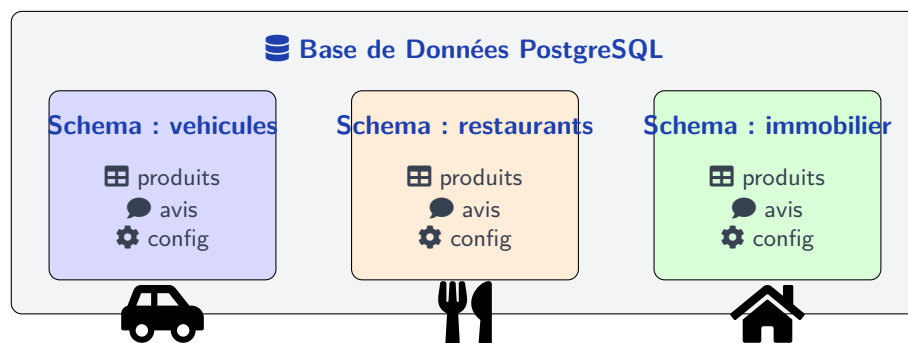


Figure 5.1. Isolation des données par schéma PostgreSQL

💡 Analogie de l'Hôtel

L'architecture multi-tenant fonctionne comme un hôtel :

- Chaque client (tenant) possède sa propre chambre (schéma)
- Les chambres sont séparées par des murs solides (isolation)

- Un client ne peut pas accéder à la chambre d'un autre
- Tous les clients utilisent le même hôtel (infrastructure partagée)

Stratégies de Modèles d'Intelligence Artificielle

Deux approches sont envisageables pour la gestion des modèles d'IA dans l'architecture multi-tenant.

6.1 Approche A : Modèle Partagé

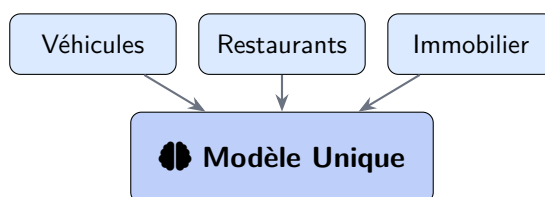


Figure 6.1. Approche A : Un modèle partagé pour tous les tenants

✓ Avantages	✗ Inconvénients
Simple à gérer et maintenir	Moins précis pour vocabulaires spécialisés
Économe en ressources (mémoire)	Pas de personnalisation par domaine
Déploiement rapide	Performance uniforme pour tous
Maintenance centralisée	

Table 6.1. Avantages et inconvénients de l'approche modèle partagé

6.2 Approche B : Modèles Spécialisés

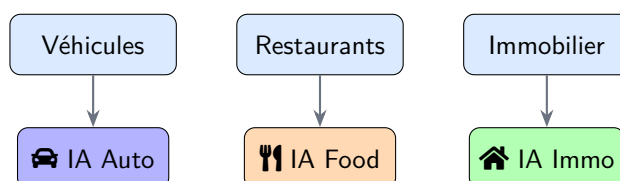


Figure 6.2. Approche B : Modèles spécialisés par domaine

✓ Avantages	✗ Inconvénients
Très précis par domaine	Plus complexe à gérer
Vocabulaire spécialisé reconnu	Consommation mémoire élevée
Possibilité de fine-tuning	Maintenance plus lourde
Personnalisation maximale	Déploiement plus long

Table 6.2. Avantages et inconvénients de l'approche modèles spécialisés

6.3 Recommandation Stratégique

💡 **Notre recommandation : Commencer simple, évoluer ensuite**

- 1 **Phase 1** : Déployer avec l'Approche A (modèle unique partagé)
- 2 **Phase 2** : Identifier les tenants nécessitant plus de précision
- 3 **Phase 3** : Ajouter des modèles spécialisés pour ces tenants spécifiques

Mécanisme de Scoring Personnalisable

7.1 Le Défi de la Personnalisation

Chaque domaine métier possède ses propres critères de qualité et de pertinence. Un système de recommandation efficace doit pouvoir s'adapter à ces spécificités.

Domaine	Critères Prioritaires
 Véhicules	Prix, kilométrage, année, marque, consommation
 Restaurants	Note globale, distance, type de cuisine, prix moyen
 Immobilier	Prix au m ² , localisation, surface, nombre de pièces
 E-commerce	Popularité, avis clients, prix, disponibilité

Table 7.1. Critères de qualité par domaine

7.2 Configuration des Poids de Scoring

Chaque plateforme peut définir ses propres critères avec des poids personnalisés.

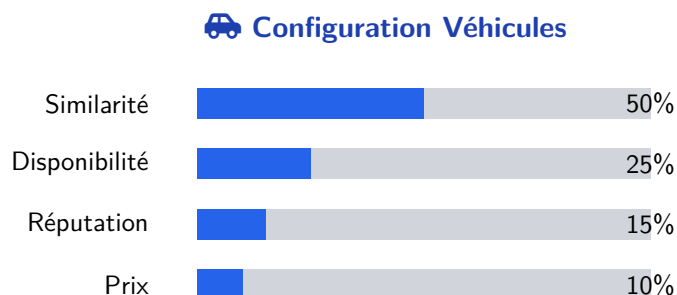


Figure 7.1. Exemple de configuration des poids pour le domaine Véhicules

7.3 Calcul du Score Final

Exemple de calcul pour un produit :

Critère	Valeur	Poids	Contribution
Similarité	0.80	× 50%	= 0.40
Disponibilité	1.00	× 25%	= 0.25
Réputation	0.80	× 15%	= 0.12
Prix	0.80	× 10%	= 0.08
Score Final :			0.85

★ Ce produit obtient un score de **85%** de pertinence !

Avantages et Bénéfices

8.1 Pour l'Entreprise

Icône	Avantage	Explication
	Économie	Un seul système à maintenir au lieu de plusieurs
	Rapidité	Nouvelle plateforme opérationnelle en quelques heures
	Scalabilité	Supporte des centaines de plateformes simultanément
	Maintenabilité	Les améliorations profitent à tous les tenants

Table 8.1. Bénéfices pour l'entreprise

8.2 Pour les Plateformes Clientes

Icône	Avantage	Explication
	Personnalisation	Critères de scoring adaptés à leur domaine
	Isolation	Données complètement séparées et sécurisées
	Monitoring	Tableau de bord dédié avec métriques
	IA de qualité	Accès aux derniers modèles sans effort

Table 8.2. Bénéfices pour les plateformes clientes

8.3 Pour les Utilisateurs Finaux

Icône	Avantage	Explication
🎯	Pertinence	Recommandations vraiment adaptées aux besoins
⚡	Rapidité	Réponses en quelques millisecondes
😊	Expérience	Meilleure découverte de produits et services

Table 8.3. Bénéfices pour les utilisateurs finaux

Feuille de Route d'Implémentation

9.1 Planning Prévisionnel

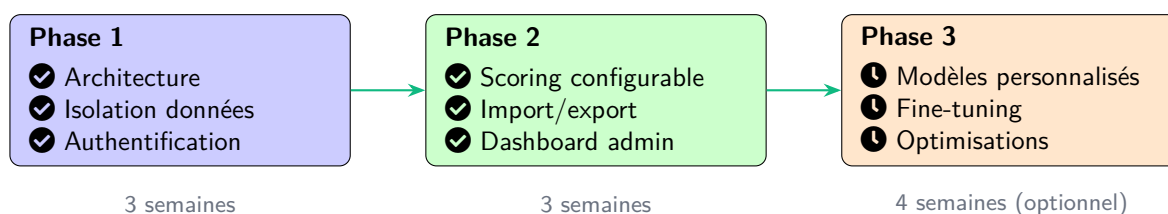


Figure 9.1. Feuille de route en trois phases

9.2 Détail des Phases

Phase	Durée	Livrables
Phase 1 Fondations	3 semaines	Architecture multi-tenant, isolation des données, middle-ware d'authentification, API d'administration
Phase 2 Fonctionna- lités	3 semaines	Moteur de scoring configurable, import/export de données, dashboard administrateur, documentation
Phase 3 Optimisa- tions	4 semaines	Support modèles personnalisés par tenant, pipeline de fine-tuning, monitoring avancé, optimisations de performance

Table 9.1. Détail des phases d'implémentation

Conclusion

L'évolution vers une architecture multi-tenant représente une opportunité stratégique majeure pour l'entreprise. Cette transformation permet de capitaliser sur les investissements existants tout en ouvrant de nouvelles perspectives de croissance.

Points Clés à Retenir

- 1 **Un système, plusieurs plateformes** : Plus besoin de recréer un système pour chaque nouveau domaine d'application
- 2 **Isolation complète** : Les données de chaque plateforme sont strictement séparées, garantissant sécurité et confidentialité
- 3 **Scoring personnalisable** : Chaque plateforme définit ses propres critères de recommandation
- 4 **IA flexible** : Choix entre modèle unique partagé ou modèles spécialisés selon les besoins
- 5 **Administration simplifiée** : Dashboard visuel pour les administrateurs et API pour les développeurs

“ En résumé

Multi-Tenant transforme un système de recommandation spécialisé en une **plateforme universelle** capable de servir n'importe quel domaine avec une **isolation totale** des données et des **critères personnalisables** par plateforme.

Glossaire

Terme	Définition
Tenant	Plateforme cliente du système (ex : “Véhicules”, “Restaurants”)
Embedding	Représentation numérique d’un texte sous forme de vecteur
Scoring	Processus de calcul du score de pertinence d’une recommandation
Qdrant	Base de données spécialisée dans la recherche de similarité vectorielle
API	Interface permettant aux applications de communiquer entre elles
Multi-tenant	Architecture où plusieurs clients partagent un même système
RaaS	Recommandation-as-a-Service, modèle de service de recommandation
Fine-tuning	Ajustement d’un modèle d’IA pour un domaine spécifique

Table 9.2. Glossaire des termes techniques